



BACKEND TECHNICAL SPECIFICATION · V1.0

MON5MAJEUR

Backend Spec for Developers

Daily fantasy NBA — French market. This document defines the business logic, the data flows and the rules the backend must implement. It is written for the incoming senior developers.

Stack (existing — keep it): Django + Django REST Framework + PostgreSQL · REST/JSON, token auth

Mobile app: unchanged — to be confirmed (likely Flutter) · **Data provider:** Goalserve (NBA package)

Audience: incoming backend developers · **Status:** ready for Phase 0

00 How to read this document

This is not a "build from scratch" brief. A substantial Django/DRF backend **already exists** and covers most of the product. Your job is to **audit it, complete the gaps, and fix the bugs** — not to rebuild it.

- **Single source of truth for endpoints & field names:** the live Swagger doc + the code. This document explains the *meaning* and the *rules*, not the exact field syntax.
- **⚠ boxes** = items to verify or decide during **Phase 0** (see consolidated checklist at the end).
- **Golden rule:** never break the existing API contract (same URLs, same response shapes) → the mobile app keeps working untouched. Any breaking change must be flagged and discussed first.

PHASE 0 — AUDIT BEFORE ANY NEW CODE

Before implementing anything, open the existing repo and deliver a short written verdict:

1. **What actually works** — functional, tested endpoints.
2. **What is missing or broken** — see the priority gaps flagged throughout.
3. **Verdict:** finish the existing backend, or rebuild (only with evidence it is unrecoverable). **Default expectation: finish the existing one.**

A Swagger page only shows the API *surface* (routes). It does NOT prove the logic behind each route is correct. Treat "endpoint exists" as "to be verified", not "done".

Three non-negotiable principles

1. **The mobile app never calls Goalserve directly.** Only our own API. The Goalserve key lives on the backend only.
2. **The backend caches Goalserve data.** The team-composition screen must load in **under 0.5s** — served from cache, never from a live external call.
3. **All business logic is computed server-side** — pricing, scoring, bonuses, lock, validation. The app only displays results. This guarantees fairness and prevents cheating.

GLOBAL CONVENTION — TIMEZONE

Every time boundary in the game uses a single reference timezone: **Europe/Paris**. This includes the daily close, the weekly/monthly resets, the price recalculation, and the composition window.

SCOPE NOTE — NOTHING ADDED, SCOPE REDUCED

Every feature in this document (notifications, OUT status, live score, bonuses, leagues...) was part of the **original agreed scope** from the start. Nothing new is introduced here. On the contrary, the scope has been **reduced** versus the initial brief (e.g. the ad system was removed) to **ship faster**. This spec describes a scope **equal to or smaller than** what was already agreed — it is a clean re-statement, not an extension.

01 Product overview

Mon5Majeur is a **daily fantasy NBA** mobile app. Each night, the user builds a 5-player team within a budget, and scores points based on the players' real performances in that night's NBA games.

The game loop

1. **Compose** — Before the night's first tip-off, the user picks 5 NBA players (from tonight's games) within a budget. Each player has a price reflecting recent form.
2. **Lock** — At the first tip-off, all teams freeze. No more edits.
3. **Score** — Each player generates a Fantasy Score from his real stats. Team score = sum of the 5 players.
4. **Rank** — The score drives the user's standing in his competitions.

The three league types

There is one cumulative world league, plus two flavours of the same duel engine. Get this distinction right — it drives much of the backend.

Type	How it works
Global League (the world)	A single world league, joined once at signup, never left . 100M fixed. No bonuses . No duels — ranking by cumulative score (weekly + monthly).
Private league (duel — by code)	Groups of friends (4–10 , even only). 1-v-1 duels each night, home-and-away championship → playoffs. Bonuses available . Joined via an invite code , not publicly listed.
Public league (duel — open)	Exactly the same duel engine as a private league (duels, home-and-away, playoffs, bonuses) — but open to everyone, no code , visible directly in "join a league". For users who don't have friends to play with.

TERMINOLOGY — DO NOT MIX UP

Global League = the world cumulative leaderboard (module `global-league`). **Public league** = an OPEN duel league (module `public-leagues`), mechanically identical to a private league. These are two different things — never conflate "global" and "public".

Key notions

- **Dynamic price** — a player's price changes daily based on recent form. Hot players cost more.
- **Tokens** — the in-app virtual currency (used for bonuses and the premium live score).
- **Fantasy Score depends ONLY on the player's own stats**. His NBA team winning or losing has zero impact on his score or price.
- **Free bonuses** in a duel league (private & public): each user gets a free bonus stock sized by the league's player count; more can be bought with tokens.

02 Data model

Reconstructed from the existing API + product logic. **To reconcile with the real code in Phase 0.**

Entity families

Family	Entities
Users & accounts	User · UserProfile (pseudo, token balance, premium/live status) · OTP / verification · Apple/Google social auth
NBA data (Goalserve cache)	Team · Player (with daily price + IN / OUT status + position) · Game (state SCHEDULED/LIVE/FINAL) · PlayerGameStats (real stats + computed Fantasy Score) · price history (last scores)
Leagues	League (global / public / private; type, budget, size, status, admin) · Membership · Match/duel (in a duel league: 2 users on one night) · Standings
Lineups	Lineup = (user + league + match_day), with its 5 starters (+ optional 6th man), used budget, locked state · LineupPlayer (player ↔ slot)
Bonuses & monetization	Bonus catalog · per-user-per-league free quota + usage · TokenPack / token transactions
Rewards & content	Jersey (user-selectable cosmetic) · static content (about, FAQ, legal, privacy)

Relations that must never break

- A Lineup = exactly 5 starters, always tied to a unique (user, league, match_day).
- In a duel league (private or public), a Match is a duel between 2 users on a given night.
- The free-bonus quota is counted **per user AND per league** (never global, never shared between members).
- A player's price is computed on his **last 5 PLAYED games** (absences excluded).
- A player's Fantasy Score depends only on his own stats.

PHASE 0 — TO CLARIFY

- Single **League** model with a **type** field (public/private) vs two separate models? Everything points to a single model — confirm in code.
- **Three league concepts:** **global-league** (world cumulative), **public-leagues** (open duel) and **private-leagues** (private duel) coexist on purpose. Just confirm they stay cleanly separated in the model — global = world cumulative, public/private = duel.

03 Existing API map

Overview of the modules already present. Detailed rules live in Chapter 4. **Source of truth: live Swagger + code.**

Module	Role	Type
--------	------	------

auth/* , dj-rest-auth/* , djoser/* , UserProfiles/*	Signup, login, OTP, accounts, profiles	standard
games-today	Tonight's NBA games (the slate)	specific
players-today	Selectable players tonight (price, status)	specific
nba/match/* , player-details , teams/*/stats	Player/team data & stats (scoring source)	specific
global-league (+ global-leagues?)	World cumulative league (the "Global League")	specific
public-leagues/*	Open duel leagues (same engine as private, no code)	specific
private-leagues/*	Private duel leagues (by invite code) — season, playoffs	specific
bonuses/*	Strategic bonuses (Curry, Luxury, 6th Man)	specific
tokens/*	Tokens & packs (monetization)	specific
leaderboard , jerseys/*	Rankings, jerseys	mixed
faqs , aboutus , legal-notice , privacy-policies	Static content & GDPR	standard

04 Business logic

This is the core of the document — everything that cannot be guessed and that caused confusion before. Read it carefully.

4.1 — Lineup selection

Before lock, the user builds a team of **exactly 5 players** from tonight's games, respecting a fixed position structure and the budget. The lineup then freezes and feeds the scoring.

Position structure — 5 fixed slots

The court has 5 fixed slots. Each slot accepts a **family** of positions.

Slot	Accepted positions (Goalserve)
Slot PG	PG, SG, or G
Slot SG	PG, SG, or G
Slot SF	SF, PF, or F
Slot PF	SF, PF, or F
Slot Center	C only

Equivalent to a **2 backcourt + 2 wings + 1 center** structure. Families: PG/SG/G → backcourt · SF/PF/F → wings · C → center.

Validation rules (always server-side)

A submission is accepted only if ALL are true; otherwise reject with `400` + clear message. Never trust the client.

1. Exactly 5 distinct players (no duplicate of the same player).
2. All 5 slots filled; each player matches his slot's family. **One player can occupy only one slot.**
3. All players play tonight (present in `players-today` for this `match_day`).
4. No `OUT` player (see 4.7).
5. Budget respected: sum of the 5 prices $\leq$ effective budget.
6. The night is not locked yet.

Multiple players from the **same NBA team are allowed without limit**.

Effective budget

- Global League: **100M fixed**. • Duel leagues (private & public): **80M or 100M** (chosen at league creation).

```
effective_budget = base_budget (80 or 100) + luxury_tax_bonus (+ playoff_seeding_bonus)
```

The 6th Man bonus adds a 6th player **out of budget** ($\leq 8M$) — handled in 4.4, not in the starters' budget.

Lock & composition window

- A **"night" (match_day) = all games of the same NBA date (US/EST)**, grouped by Goalserve's `date` field — NOT by Paris calendar day. This keeps early games and late West-coast games of the same NBA night together (one NBA date spans roughly Sun 19:00 → Mon 06:00 Paris). Grouping by Paris day would split a night in two — wrong.
- **Early games count.** On some days (e.g. Sundays) the NBA schedules early games — a tip-off as early as ~19:30 Paris. They belong to the night; the lock is simply that first tip-off (19:30). The composition window still opens at that morning's 09:00 close → a 09:00 → 19:30 window. (NBA games never start before 09:00 Paris, so the lock is always after the composition opens.)
- The lock is **global for the night**, triggered at the **first game's tip-off** (earliest `datetime_utc` in `games-today`). **This time is variable every day** — never a fixed hour. Depending on the NBA schedule the first game may start at 19:30, midnight, 1am, 2am... (Paris); the lock simply follows the earliest game.
- **Composition window: opens at 09:00 (Paris) → closes at the lock** (the night's first tip-off, variable). Compositions open at 09:00 with freshly recalculated prices. Between the lock and the next 09:00 close, composition is closed — users never compose on stale prices.
- Before lock: unlimited edits; only the last valid submission counts (it overwrites the previous).
- At/after lock: any submission refused, for all leagues, even for players whose game starts later.
- **No lineup submitted before lock → forfeit, 0 points that night.** In a duel league, the night's duel is played with a 0 (loss if the opponent submitted).

```
SLOTS = {
    "PG":      {"PG", "SG", "G"},
    "SG":      {"PG", "SG", "G"},
    "SF":      {"SF", "PF", "F"},
    "PF":      {"SF", "PF", "F"},
    "CENTER":  {"C"},
}
```

```
def validate_lineup(user, league, match_day, slot_assignments):
    # slot_assignments = { "PG": player_id, "SG": ..., "SF": ..., "PF": ..., "CENTER": ... }
```

```

if league_is_locked(match_day):
    reject("Night locked")
if set(slot_assignments) != set(SLOTS):
    reject("All 5 slots must be filled")
ids = list(slot_assignments.values())
if len(set(ids)) != 5:
    reject("A player can occupy only one slot")

for slot, pid in slot_assignments.items():
    p = get_player(pid, match_day)
    if not p.plays_tonight:
        reject(f"{p.name} does not play tonight")
    if p.status == "OUT":
        reject(f"{p.name} is OUT")
    if p.position not in SLOTS[slot]:
        reject(f"{p.name} cannot fill slot {slot}")

players = [get_player(pid, match_day) for pid in ids]
if sum(p.price for p in players) > effective_budget(user, league, match_day):
    reject("Budget exceeded")
save_lineup(user, league, match_day, slot_assignments) # overwrites previous

```

PHASE 0

- Position comes from the Goalserve **roster** feed (PG/SG/SF/PF/C, sometimes generic G/F). Verify the backend fetches/stores it per player; define a fallback if `position` is empty.
- Confirm "one lineup per league per night" (the API suggests it) vs a shared lineup.

4.2 — Fantasy Score (the critical section)

Each player, per game, generates a Fantasy Score: raw stats + a shooting-efficiency bonus/malus, ×0.7.

$$\text{Fantasy Score} = \text{round}(\text{Base} + \text{Efficiency}) \times 0.7 \quad \# \text{ round ONLY at the very end}$$

DESIGN INTENT — READ BEFORE CODING

Points are counted **twice**: once in Base (via PTS), once in Efficiency (per made/missed shot). **This is intentional, do NOT de-duplicate.** It separates two players who score the same points with different efficiency: 30 pts on 12 shots must beat 30 pts on 40 shots (many misses → big efficiency malus). The system rewards efficiency and penalises volume waste.

Block 1 — Base

$$\text{Base} = \text{PTS} + \text{REB} + \text{AST} + \text{STL} + \text{BLK} - \text{TO}$$

Element	Value	Goalserve field
Point (PTS)	+1 each	points
Rebound (REB)	+1	total_rebounds
Assist (AST)	+1	assists
Steal (STL)	+1	steals
Block (BLK)	+1	blocks
Turnover (TO)	-1	turnovers

Block 2 — Shooting efficiency (where bugs happen)

Shot	Made	Missed
3-point	+2	-2
2-point	+1	-1
Free throw	+0.5	-0.5

GOALSERVE DOES NOT GIVE 2-POINTERS — DERIVE THEM

Goalserve provides (per player): `field_goals_made/attempts` = ALL field goals (2pt + 3pt combined), `threepoint_goals_made/attempts` = 3pt only, `freethrows_goals_made/attempts` = FT. **2-pointers do not exist as a field — compute them by subtraction:**

$$\begin{aligned} 2\text{pt_made} &= \text{field_goals_made} - \text{threepoint_goals_made} \\ 2\text{pt_att} &= \text{field_goals_attempts} - \text{threepoint_goals_attempts} \\ 2\text{pt_missed} &= 2\text{pt_att} - 2\text{pt_made} \end{aligned}$$


```
3pt_missed = threepoint_goals_attempts - threepoint_goals_made
ft_missed = freethrows_goals_attempts - freethrows_goals_made
```

⊘ **NEVER** use the Goalserve **two_pt** field — it is an American-football field ("two-point conversions") from the multi-sport template. Using it = guaranteed bug.

```
Efficiency = (3pt_made × +2) + (3pt_missed × -2)
            + (2pt_made × +1) + (2pt_missed × -1)
            + (ft_made × +0.5) + (ft_missed × -0.5)
```

0/0 case: if a player attempted no shot of a type, made = missed = 0 → contribution 0. Always work in made/missed, never in percentages.

Block 3 — Assembly

```
raw = Base + Efficiency          # keep decimals (FT give ±0.5)
Fantasy Score = round(raw × 0.7) # SINGLE rounding, at the very end
```

Worked examples

Player / line	Base	Efficiency	Raw	×0.7 → FP
SGA: 31p 2r 8a 0s 1b 3to · FG 10/17 · 3pt 2/4 · FT 9/11	39	+6.5	45.5	32
TO-heavy: 18p 2r 6a 1s 2b 7to · FG 8/15 · 3pt 0/1 · FT 2/3	22	+0.5	22.5	16
Big, no 3pt: 5p 10r 5a 3s 0b 3to · FG 2/4 · 3pt 0/0 · FT 1/2	20	0	20	14

```
def fantasy_score(p):    # p = Goalserve <player> block
    base = p.points + p.total_rebounds + p.assists + p.steals + p.blocks - p.turnovers
    fg2m      = p.field_goals_made      - p.threepoint_goals_made
    fg2_missed = (p.field_goals_attempts - p.threepoint_goals_attempts) - fg2m
    fg3_missed = p.threepoint_goals_attempts - p.threepoint_goals_made
    ft_missed  = p.freethrows_goals_attempts - p.freethrows_goals_made
    eff = (p.threepoint_goals_made*2 - fg3_missed*2) \
          + (fg2m*1 - fg2_missed*1) \
          + (p.freethrows_goals_made*0.5 - ft_missed*0.5)
    return round((base + eff) * 0.7)          # single rounding
# ⊘ never use p.two_pt (football field)
```

4.3 — Player pricing (dynamic)

A player's price changes each night with his recent form.

$$\text{Price} = \text{round}(\text{Weighted_average} \div \text{Zone_divisor})$$

DEBUG SCORES BEFORE PRICES

Price is computed FROM the per-game Fantasy Scores (4.2). If a price is wrong, the cause is almost always upstream in the score (especially the 2pt derivation). Validate a player's per-game scores are correct **before** looking for a bug in the pricing formula. Correct pricing on wrong scores still yields wrong prices.

Inputs

- The player's **last 5 PLAYED games**. A non-played game (absence → 0) is NOT counted; go back to the previous played game.
- Each input = that game's **final rounded Fantasy Score** (the integer FP), not the decimal raw.

Step 1 — Weighted average

Score position (of the 5)	Coefficient
Best	×1
Worst	×1
3 middle	×2 each
Divisor	÷8

Coefficients sum to 8 → a true normalised average. It dampens extremes (a hot night / a disaster weigh less) and weights the "usual level" (the 3 middle scores). Ties: sort ascending — the result is identical regardless of which equal value is "worst" vs "middle".

Step 2 — Zone divisor

Zone	Weighted average	Divisor	Profile
A	≤ 16	÷ 1.2	Bench
B	> 16 and ≤ 25	÷ 1.1	Starters
C	> 25	÷ 1.0	Superstars

Boundaries are continuous — no gap. A bigger divisor lowers the price at equal FP, so bench players are made proportionally cheaper and superstars proportionally pricier (prevents star-stacking). **Do not collapse to a single divisor — it would break game balance.**

Step 3 — Recalculation timing

Prices are recomputed once per day at the **09:00 daily close** (see 5.2), after the previous night's final scores are confirmed and before the next lock. Once computed they are **frozen and identical for everyone** for that slate. An absent player's price does not move (no new played game).

Worked example (SGA → 22M)

Last-5 FP: 16 · 19 · 23 · 31 · 32 → (best 32×1)+(worst 16×1)+(19+23+31)×2 = 194 ; 194÷8 = 24.25 ; zone B ÷1.1 → 24.25/1.1 = 22.04 → **22M**.

Fewer than 5 played games (normal at launch — October = season start)

During the first weeks almost no player has 5 games. This is the **default state at launch**, not a rare edge case.

Games played	Calculation
≥ 5	Normal: $\text{best} \times 1, \text{worst} \times 1, 3 \text{ middle} \times 2, \div 8$
4	$\text{best} \times 1, \text{worst} \times 1, 2 \text{ middle} \times 2 \rightarrow \div 6$
3	$\text{best} \times 1, \text{worst} \times 1, 1 \text{ middle} \times 2 \rightarrow \div 4$
2	simple average $\div 2$
1	that single game's score
0	floor price = 3M (returned directly, no zone)

For ≥ 3 games the divisor is the sum of coefficients actually used (8 at 5 games, 6 at 4, 4 at 3). The zone divisor still applies after the weighted average — except the 0-game case which returns 3M directly.

```
FLOOR_PRICE = 3    # M, for players with no history

def player_price(played_fp):    # final Fantasy Scores of PLAYED games (5 most recent)
    n = len(played_fp)
    if n == 0: return FLOOR_PRICE
    s = sorted(played_fp)
    if n == 1:    weighted_avg = s[0]
    elif n == 2: weighted_avg = (s[0] + s[1]) / 2
    else:
        # 3..5
        coef_sum = 1 + 1 + 2 * len(s[1:-1])    # 4 / 6 / 8
        weighted_avg = (s[0] + s[-1] + sum(s[1:-1]) * 2) / coef_sum
    divisor = 1.2 if weighted_avg <= 16 else (1.1 if weighted_avg <= 25 else 1.0)
    return round(weighted_avg / divisor)
```

4.4 — Strategic bonuses (duel leagues — private & public)

Available in duel leagues only (private AND public — they share the same engine). Not available in the Global League. Activated for a given night/duel, before lock. Quota/tokens checked server-side at activation.

Free quota — per user, per league

Each user gets his own free stock in each duel league, sized by league size. **One-shot** (granted on entry, does NOT refill). Beyond the quota → buy per unit with tokens.

League size	Chef Curry	Luxury Tax	6th Man
4	1	1	1
6	2	1	2
8	2	2	2
10	3	3	3

Luxury Tax

+5M budget for the night. Added to `effective_budget` before lineup validation.

Chef Curry

+3 points added to the user's **final duel score** (the "clutch Curry three"). Applied AFTER summing the 5 players, to this user's duel score only. Does not touch individual scores, prices, or the Global League.

6th Man

- **At selection:** add a 6th player, price $\leq 8M$ **strict**, any position, **out of budget**. He must also play tonight and not be OUT. The 5 starters still respect the 2+2+1 structure; the 6th man is out of structure.
- **At scoring (end of night):** compute all 6 scores; team score = **sum of the top 5 of the 6**, regardless of position. The weakest of the 6 is dropped.
- **Live:** show all 6 live scores + the continuously recomputed top-5 total.

Example: starters 30,25,20,15,**10** · 6th man 22 → keep 30,25,22,20,15 = 112 (the 10 is dropped). If the 6th scored 8 (< worst starter), the 5 starters stay and the 6th is unused.

```
def duel_score(lineup, bonuses):
    scores = [fantasy_score(p) for p in lineup.starters]      # 5
    if bonuses.sixth_man:
        scores.append(fantasy_score(lineup.sixth_man))
        scores = sorted(scores, reverse=True)[:5]            # top 5 of 6
    total = sum(scores)
    if bonuses.chef_curry: total += 3
    return total
# Luxury Tax does NOT appear here — it acts earlier, on the selection budget.
```

APPLICATION ORDER

Luxury Tax → before lock (selection budget). 6th Man & Chef Curry → at scoring (end of night / live). Do not mix them.

4.5 — Live score (premium)

Premium users see their players' Fantasy Scores update in near real time during games. This is the paid feature at launch. **No live endpoint is visible in the current API → likely to build.**

Access — tokens only

- Live access is bought **exclusively with tokens** (module `tokens`), as a time-limited access. Backend stores `premium_until` per user.
- **Checked on EVERY live request:** `now < premium_until` ? If not → `403`. A non-premium user must never get live data, even by calling the API directly. Server-side check, never client.

Coverage & payload

- Covers **everything**: the Global League + all the user's duel leagues (public & private), all active lineups tonight.
- Returns, per lineup: live Fantasy Score per player (computed with the 4.2 formula — never raw stats alone), team total (with bonuses), each game's state (`SCHEDULED/LIVE/FINAL`), in a duel league the opponent's live total, plus a freshness timestamp + `is_stale` flag.
- 6th Man: show all 6 live + the continuously recomputed top-5 total.

Refresh

- During games: refresh **every 1 minute** (LIVE games only).
- One **final refresh with official stats** at the 09:00 close → the official final score **overwrites** the live one and becomes the truth for standings, final scoring and pricing.

Edge cases

- Goalserve live down/late → serve last cached score + `is_stale: true`. Never show 0, never crash.
- Premium expires mid-game → access cut on the next request.
- OUT player live → 0 (consistent with 4.7). · No LIVE game → serve frozen FINAL scores.

PHASE 0

- Confirm/ build the live endpoint. · Validate the Goalserve quota supports "1 refresh/min during games" on the current package. · Confirm the premium purchase mechanism in `tokens` (price, duration).

4.6 — Leagues

4.6.1 — Global League (the world)

A single world league, joined at signup, **never left**. 100M fixed. **No bonuses**. No duels — ranking by **cumulative score**. (Module: `global-league` . Do not confuse with the Public league, which is a duel league — see 4.6.2.)

- Each night, the lineup score (sum of 5, no bonus) is **added** to the user's totals. No lineup → 0 added.
- **V1 periods (Europe/Paris)**: Weekly (Mon 00:00 → Sun 23:59, reset Monday) and Monthly (1st → last day, reset on the 1st). The two counters run in parallel. **No seasonal ranking in V1**.
- Rewards: Top 8 weekly → an in-game bonus usable in duel leagues. Monthly winner → an NBA jersey.

4.6.2 — Duel leagues (private & public)

PRIVATE VS PUBLIC — SAME ENGINE, ONE DIFFERENCE

A **public league** and a **private league** run the **exact same duel engine**: even size 4–10, home-and-away round robin, duels, standings, bonuses, playoffs. The **only** difference is how you get in:

- **Private** — joined via an **invite code**; not publicly listed.
- **Public** — **open to everyone, no code**; listed directly in "join a league" (for users without friends to invite).

Everything below applies to **both**. Implement the engine once; private/public is just a flag (`is_open`) + the join path. Both are created, managed and started by a **creator (admin)** via a Waiting Room.

Lifecycle (identical for private & public)

1. **Create** — the creator sets logo, name, optional description, **budget (80 or 100M)**, **size (4/6/8/10)**, format (Head-to-Head). *Private* → an **invite code** is generated. *Public* → the league is **listed openly** in "join a league" (no code). The creator is the admin.
2. **Waiting Room** (league status `WAITING`) — the league waits, showing "X / size teams joined". Members join (private: code; public: from the open list). The creator can **Edit** settings, **Kick** members, or **Delete** the league. Nothing has started yet.
3. **Start** — **only the creator** triggers `start_league`, manually (**no auto-start**). To launch: at least **4 teams** AND an **even number**. You may start **below max capacity** (e.g. start with 6 in a league created for 8) — the schedule is built from the members present at that moment. A league **not started within 7 days is auto-deleted**.
4. **Regular season** → **playoffs** → **winner**. Once started, the roster and schedule are **frozen: no join, no leave, no kick** until the league ends.

Schedule — firm rules

- **Even size only (4, 6, 8, 10)**. Starting with an odd number is refused (clear message). This removes "bye" handling entirely — everyone plays every night.
- **Schedule frozen at `start_league`**, from members present. **No join / leave / kick after start**.
- **Home-and-away round robin**: everyone faces everyone twice (1 home, 1 away). Each NBA night = one `match_day`; all members are paired into 1-v-1 duels.

Size	Duels / night	Regular-season match days
4	2	6
6	3	10
8	4	14
10	5	18

```
def generate_schedule(members):          # members: even length (4,6,8,10)
    n = len(members); assert n % 2 == 0
    players = list(members); aller = []
    for r in range(n - 1):                # ALLER: n-1 rounds
        pairings = []
        for i in range(n // 2):
            home, away = players[i], players[n - 1 - i]
```

```

        if r % 2 == 1: home, away = away, home    # alternate home/away
        pairings.append({"home": home, "away": away})
    aller.append(pairings)

    players = [players[0]] + [players[-1]] + players[1:-1]    # circle rotation
    retour = [[{"home": m["away"], "away": m["home"]} for m in rnd] for rnd in aller]
    return aller + retour                                     # 2*(n-1) match days

```

Duel resolution

- Each night: your duel score (sum of 5 + bonuses, see 4.4) vs your opponent's. **Higher score wins.**
- **Tie → the HOME player wins.** Forfeit (no lineup) → 0 → loss. Both forfeit (0–0) → tie → home wins. **There are never draws.**

```

def resolve_duel(d):
    h, a = duel_score(d.home_lineup, d.home_bonuses), duel_score(d.away_lineup, d.away_bonuses)
    winner = d.home if h >= a else d.away    # tie → home
    return winner, h, a

```

Standings

Primary criterion: **number of wins** (like the NBA standings). Tie-break cascade, in order:

1. **Point differential** (total Fantasy points scored – conceded over the regular season).
2. **Total Fantasy points scored** (a simple number — checked before the costlier head-to-head).
3. **Head-to-head** between the tied players.
4. **Last resort:** alphabetical by pseudo (neutral, deterministic — never leave two players strictly tied).

Maintain per user/league: wins, losses, points for, points against, head-to-head results. Head-to-head is not a plain sort key — apply it as a sub-step between players still tied after the first two criteria (which is rare, so it is seldom needed).

4.6.3 — Playoffs

- **Top 4** of the regular standings qualify, seeded 1–4 by regular rank. In a 4-player league, all 4 qualify (the season just sets the 1–4 seeding for matchups + budget bonus).
- **Semifinals:** 1 vs 4 and 2 vs 3. **Final:** the two semi winners.
- Each round is **BO3** (first to 2 game wins). A tie in a single game → home player wins that game.

BO3 game	Home
Game 1	Higher seed
Game 2	Lower seed
Game 3 (if needed)	Higher seed

Seeding budget bonus (all playoff games)

Regular rank	1st	2nd	3rd	4th
Budget bonus	+4M	+2M	+1M	+0M

```
def playoff_home(game_no, higher_seed, lower_seed):  
    return lower_seed if game_no == 2 else higher_seed  
  
def playoff_budget_bonus(regular_rank):    # 1..4  
    return {1: 4, 2: 2, 3: 1, 4: 0}[regular_rank]    # added to effective_budget
```

PHASE 0

Verify the existing `playoff-qualifiers` and `winner` endpoints implement this format (Top 4, BO3, home rules, budget bonus) or adjust them.

4.7 — Player OUT status

A user must never be able to select a player who is not playing tonight. **No status endpoint is visible in the current API → build it or expose it.**

Binary model in V1: IN or OUT

Source: Goalserve injuries feed. Important: that feed stores the injury *type* in `status` (Hamstring, Knee...) and the *severity* in `description` (Sidelined, Game Time Decision...). There is **no plain "Out" field**. Detect by **keywords**, not exact equality:

Goalserve signal	V1 status	Selectable?
<code>description</code> contains "sidelined" / "will not play" / "out"	OUT	No
"Game Time Decision" / "Questionable" / other	IN	Yes
Player absent from the injuries feed	IN	Yes (default)

No Doubtful/Questionable surfaced in V1: only an explicitly "will not play" player is blocked.

Behaviour

1. **OUT = not selectable.** In `players-today`, returned with `is_selectable: false`, `status: "OUT"`. The app greys him out.
2. **Server-side double validation.** The backend rejects any lineup (starter OR 6th man) containing an OUT player (400 + clear message).
3. **Notification** if a player already in a user's lineup turns OUT before lock → notify the user to fix his team before lock.
4. **No automatic replacement.** The backend never replaces an OUT player and never edits the lineup. If the user does not fix it before lock, the OUT player stays and scores 0. (The only replacement is the user's active choice via the 6th Man bonus.)
5. **Pricing consistency:** an absence (non-played game) is excluded from the player's price (4.3).

Edge cases

- Goalserve down at refresh → keep last known cached status (don't block the whole selection).
- OUT then re-listed active same night → last known status wins (back to IN).

```
OUT_KEYWORDS = ("sidelined", "will not play", "out")
```

```
def map_goalserve_injury(report):
    desc = (report.description or "").lower()
    return "OUT" if any(k in desc for k in OUT_KEYWORDS) else "IN"
    # player absent from feed → IN (default)
```

```
def on_player_marked_out(player_id, match_day):
    for lineup in lineups_containing(player_id, match_day, locked=False):
        notify_user(lineup.user, f"{player_name(player_id)} is OUT — fix your team before lock
    # NO automatic replacement
```

- Observe the real `description` values over a few days and freeze the exact "will not play" keyword list.
- **Plan B** if the injuries feed is too unstable: cross-check official lineups near tip-off (a player absent from the night's roster = OUT).

4.8 — Notifications (push)

Push notifications via Firebase Cloud Messaging (see 5.3). They drive retention — the only S1 KPI — so they must be sharp, personal, and never spammy.

GOLDEN RULE — NEVER SPOIL

A notification **never reveals a score or a duel result**. It names the stake (the opponent, "ranking updated") to make the user open the app, but the content is discovered **only inside the app**. This protects the paid live-score feature and preserves engagement.

Also: **never a burst**. One push per key moment; if several leagues qualify, send a single prioritised push — **private league takes priority over public**.

#	Trigger	Message (private prioritised)
1	Player OUT — a player in a non-locked lineup turns OUT (see 4.7)	"{Player} is OUT — fix your team before lock."
2	Results — at the 09:00 daily close (one push, prioritised)	Private: "🏀 Your duel vs {opponent} is over — come see the result." Public (if no private league): "🏀 Standings updated — come see where you rank."
3	Composition reminder — at 19:00 (Paris) , only if the user has NOT composed yet (one push, prioritised)	Private: "Tonight you face {opponent}. Don't let them trash-talk you — set your lineup 🔥 " Public: "Don't forget your lineup tonight 🏀"

Notifs 2 & 3 are personalised (the opponent is named — it creates tension) but reveal no score/result. The 19:00 reminder fires only for users with an empty lineup for that night; if a user already composed, no reminder.

PHASE 0

Confirm FCM is wired. The 19:00 reminder needs a scheduled job that checks, per user, whether a lineup exists for tonight (across the user's leagues, private first). Results notif is part of the 09:00 close job.

05 Integrations, scheduling & ops

5.1 — Goalserve integration (cache)

The backend is the only caller of Goalserve. It fetches on a schedule, caches in the DB, and serves the app from cache only. Key on the server only.

Feed	Used for	Refresh
Rosters (teams + players + positions)	Slots (2+2+1), teams	1×/day
Schedule / games	<code>games-today</code> , first tip-off (lock)	1×/day + adjust
Injuries	<code>IN</code> / <code>OUT</code> status (4.7)	Frequent
Live / in-game stats	Live score (4.5)	Every 1 min during games
Results / box scores	Final scoring (4.2), pricing (4.3)	After each night

Resilience: if a feed is down at refresh, keep the last cached value (never an empty screen, never a blocked selection). `is_stale` flag where relevant.

5.2 — Scheduled tasks (CRON) — the engine

TIMEZONE TRAP

NBA games are played in the US evening = **late night / early morning Paris time** (the slate can begin around midnight and run to ~09:00, and the exact times vary every day). So the price "midnight" cannot be a naive 00:00 — it is anchored to the NBA cycle. Logical rule: **a night's prices are computed AFTER the previous night's final scores are confirmed and BEFORE the next lock; once computed, frozen and identical for everyone.**

The 09:00 daily close (Europe/Paris) — results published to everyone at once

Time (Paris)	Task
Late night → morning	NBA games (typically ~midnight–09:00 Paris, varies nightly): live score (1-min refresh) for premium · scoring computed in the background
09:00	Daily close: freeze final scores (4.2) → resolve duels + standings (4.6) → publish results to all → recompute prices (4.3) for the next slate
Mon 09:00	+ weekly reset & Top-8 reward (after close)
1st of month 09:00	+ monthly reset & jersey reward (after close)
During the day	Injury-status refresh (frequent), <code>games-today</code> / <code>players-today</code> refresh
19:00	Composition reminder push — only to users without a lineup for tonight (see 4.8)

First tip-off (variable)	Lock for that night — at the earliest game's tip-off, which changes every day (could be midnight, 1am, 2am... Paris)
-----------------------------	---

All users see results at 09:00 — same time for everyone (fairness + a daily ritual). Resets are at 09:00 (not 00:00) so they happen WITH the results publication.

COMPOSITION IS CLOSED BETWEEN THE FIRST TIP-OFF AND THE 09:00 RESULTS

Once the night's first game tips off (lock), composition is **closed** and stays closed until the next 09:00 close publishes results and reopens it. In other words: **during the games and overnight, no one can build or edit a lineup**. The window is strictly 09:00 → first tip-off.

STRICT DEPENDENCY ORDER AT 09:00

Final scores → Duels & standings → Publish results → Recompute prices

Prices and standings both depend on final scores. Never recompute prices before the night's scores are confirmed.

PHASE 0

Validate Goalserve final stats are reliable/complete by 09:00 (West-coast games end late). If some data arrives after 09:00, plan a follow-up update. Verify the backend already blocks `join` after `start_league` and refuses an odd-numbered start.

5.3 — Security, GDPR & monitoring

Security (mostly verify the existing Django)

- Goalserve key server-side only. · Passwords hashed (never plaintext). · All game logic server-side. · Endpoints token-protected (verify none sensitive is open). · Validate all inputs server-side (lineup, token purchases, bonus activation).

GDPR (mandatory — store publishing condition)

- Inventory personal data collected: email, phone (OTP), pseudo, game history, token purchases. Document where stored and who has access.
- Privacy policy + legal notices reachable in-app (`privacy-policies` , `legal-notices` endpoints exist — fill & wire them).
- **Account deletion** must erase ALL data (profile, lineups, history, payment-linked data), not just deactivate. `djoser users/me` DELETE exists — verify it is complete.
- Consent to data processing at signup.

Monitoring — the solo-founder safety net

Need	Tool	Side
Retention D7/D30 (key S1 signal)	Firebase Analytics	App
App crashes	Firebase Crashlytics	App
Push notifications (OUT player, 09:00 results)	Firebase Cloud Messaging	App
Backend errors	Sentry	Server
Server down / unresponsive	Uptime check (e.g. UptimeRobot)	Server
09:00 CRON failure	Dedicated alert	Server

Priority alerts (to the founder, email/SMS): ① 09:00 CRON failed (results not published — urgent) · ② server unreachable on a game night · ③ live score down during games (paying premium impacted) · ④ abnormal app crash rate.

Firebase watches the app, NOT the server. The most dangerous solo scenario (server/CRON down overnight) is a server-side blind spot → Sentry + uptime + the dedicated 09:00 alert are essential.

A Phase 0 — Consolidated audit checklist

Run all of this before writing new code. Deliver a short written verdict.

Architecture & data

- ☐ Confirm stack (Django/DRF) & database (PostgreSQL?).
- ☐ Single `League` model with a `type` field, or two models?
- ☐ Confirm the three league types are cleanly modeled: `global-league` (world cumulative), `public-leagues` (open duel), `private-leagues` (private duel).
- ☐ Confirm "one lineup per league per night" vs shared.
- ☐ Verify mobile app stack (likely Flutter) and that the API contract it expects matches.

Goalserve & data correctness

- ☐ Player positions fetched/stored from the roster feed; fallback if empty.
- ☐ 2-pointers derived by subtraction (never the `two_pt` football field).
- ☐ Validate the Fantasy Score on a few real players, per game (root-cause of the price discrepancy).
- ☐ Injuries: freeze the real "will not play" keyword list; Plan B = official lineups near tip-off.
- ☐ Goalserve quota supports 1 live refresh/min during games.
- ☐ Final stats reliable/complete by 09:00; follow-up plan for late data.

Gaps likely to build

- ☐ Player OUT status endpoint & selection blocking.
- ☐ Live score endpoint + premium check (`403`) + token purchase mechanism.
- ☐ Verify pricing CRON, scoring CRON, status refresh exist & are correct.
- ☐ Verify bonus effects (Luxury Tax, Chef Curry, 6th Man) are actually computed server-side.
- ☐ Verify `join` blocked after `start_league` + odd-numbered start refused.
- ☐ Verify `playoff-qualifiers` / `winner` match the Top4 / BO3 / budget-bonus rules.
- ☐ OTP → persistent session (token strategy, refresh) so users stay logged in.

Security / GDPR / ops

- ☐ No endpoint open without auth; all game logic server-side.
 - ☐ Account deletion erases everything (not just deactivate).
 - ☐ Privacy policy / legal notices wired & compliant.
 - ☐ Server monitoring (Sentry + uptime + 09:00 CRON alert) in place.
-

Canonical values: Fantasy multiplier $\times 0.7$ · price zones $\leq 16 \div 1.2$ / $> 16 \leq 25 \div 1.1$ / $> 25 \div 1.0$ · playoff budget bonus +4/+2/+1/+0 · daily close 09:00 Europe/Paris.